OPERATING SYSTEM

Complete Notes – All Major Chapters

Simple English | Definitions | Key Concepts | Examples | Exam Revision

For BCA / MCA / Computer Science Students

Study note: Chapter names and depth can vary by university syllabus. Use these notes as a broad study guide and compare them with your official syllabus.

Chapter 1: Introduction to Operating System

An **Operating System (OS)** is system software that manages computer hardware and software resources and provides services to application programs. Examples: Windows, Linux, macOS, Android.

Objectives of an OS

- **Convenience:** makes the computer easier to use.
- **Efficiency:** uses CPU, memory, storage and devices effectively.
- **Resource management:** allocates resources among programs and users.
- **Protection and security:** helps prevent unauthorized access.

Main functions of an OS

- Process management – creates, schedules and terminates processes.
- Memory management – allocates and frees main memory.
- File management – creates, reads, writes and organizes files and folders.
- Device and I/O management – controls input/output devices through drivers.
- Security and protection – manages permissions and access.
- User interface – provides a command-line interface (CLI) or graphical user interface (GUI).
- Error handling, accounting and networking services.

Types of Operating Systems

- **Batch OS:** collects jobs and processes them in batches, with little or no direct user interaction.
- **Multiprogramming OS:** keeps several jobs in memory so the CPU can work on another when one waits for I/O.
- **Time-sharing / Multitasking OS:** shares CPU time among tasks or users for interactive response.
- **Multiprocessing OS:** uses two or more processors/CPU cores.
- **Real-Time OS (RTOS):** responds within specified timing constraints; used in control and embedded systems.
- **Distributed OS:** coordinates resources across multiple networked computers.
- **Network OS:** provides network services such as shared files, printers and user management.
- **Embedded / Mobile OS:** designed for dedicated devices or mobile devices.

System calls and OS services

A **system call** is a programmed request for a service from the OS kernel. Common categories include process control, file management, device management, information maintenance, communication and protection. Examples on Unix-like systems include fork(), exec(), read(), write() and open().

Kernel vs Shell

- **Kernel:** core part of the OS; manages CPU, memory, devices and system resources.
- **Shell:** interface that accepts user commands; it may be command-line or graphical.

Chapter 2: Process Management

A **program** is a passive set of instructions stored on disk. A **process** is a program in execution, including its current state and resources.

Process states

- **New:** process is being created.
- **Ready:** waiting to be assigned the CPU.

- **Running:** instructions are executing on the CPU.
- **Waiting / Blocked:** waiting for an event, such as I/O completion.
- **Terminated:** process has finished or has been stopped.

Process Control Block (PCB)

The PCB is an OS data structure that stores information about a process: process ID, process state, program counter, CPU registers, scheduling information, memory information, accounting information and I/O status.

Context switching

A context switch occurs when the OS saves the state of one process/thread and restores another's state so the CPU can switch execution. It enables multitasking but adds overhead.

Threads

A thread is a basic unit of CPU execution within a process. Threads of the same process generally share code, data and open resources, while each has its own stack and CPU register state.

- **User-level threads:** managed by a user-space library; can be fast to switch, but behavior depends on the threading model.
- **Kernel-level threads:** managed by the OS; can support parallel execution on multiple cores, with kernel management overhead.

Process vs Thread

Process	Thread
Has its own process address space and resources.	Usually shares the process address space and resources.
Creation and switching are generally more costly.	Creation and switching are often less costly.
Processes are isolated from each other more strongly.	A faulty thread can affect other threads in the same process.

Inter-process communication (IPC)

- **Shared memory:** processes access a common memory region; synchronization is needed.
- **Message passing:** processes exchange messages through OS-supported mechanisms.
- Other IPC mechanisms include pipes, sockets, signals and message queues.

Chapter 3: CPU Scheduling

CPU scheduling selects a process from the ready queue and assigns the CPU to it.

Scheduling goals

- Maximize CPU utilization and throughput.
- Minimize waiting time, turnaround time and response time.
- Provide fairness and avoid starvation.

Important terms

- **Arrival Time (AT):** time a process enters the ready queue.
- **Burst Time (BT):** CPU time required by a process.
- **Completion Time (CT):** time a process finishes.
- **Turnaround Time (TAT) = CT** – **AT.**
- **Waiting Time (WT) = TAT** – **BT** in the basic single-CPU scheduling problems without separately counted I/O.

- **Response Time:** first CPU start time – arrival time.

Scheduling algorithms

- **FCFS (First Come, First Served):** runs processes in arrival order; simple but may cause the convoy effect.
- **SJF (Shortest Job First):** selects the shortest next CPU burst; can minimize average waiting time when burst lengths are known, but long jobs may starve.
- **SRTF (Shortest Remaining Time First):** preemptive form of SJF.
- **Priority Scheduling:** highest-priority process runs first; starvation can be reduced by aging.
- **Round Robin (RR):** each ready process gets a time quantum; useful for interactive systems.
- **Multilevel Queue:** separates processes into fixed queues with different policies.
- **Multilevel Feedback Queue:** allows processes to move between queues.

Preemptive vs Non-preemptive

Preemptive: the OS may take the CPU away from a running process. **Non-preemptive:** a process keeps the CPU until it finishes its CPU burst or blocks.

Numerical problem method

- Write AT and BT for each process.
- Draw the Gantt chart using the given scheduling rule.
- Read CT from the chart.
- Calculate TAT = CT – AT and WT = TAT – BT.
- Average waiting time = total WT ÷ number of processes.

Chapter 4: Process Synchronization

Synchronization coordinates concurrent processes or threads that access shared data.

Race condition and critical section

A **race condition** occurs when the result depends on the timing/order of concurrent operations. A **critical section** is the part of code that accesses shared data or resources.

Requirements for a correct critical-section solution

- **Mutual exclusion:** only one process at a time enters the critical section for the same protected resource.
- **Progress:** if the section is free and processes want to enter, choosing the next entrant should not be postponed indefinitely.
- **Bounded waiting:** a process should not wait forever after requesting entry.

Synchronization tools

- **Mutex:** lock that allows one thread at a time to access a protected section.
- **Semaphore:** integer synchronization primitive used with atomic wait and signal operations.
- **Binary semaphore:** commonly takes values 0 or 1.
- **Counting semaphore:** tracks multiple available instances of a resource.
- **Monitor:** higher-level construct that combines shared data and synchronized procedures.

Classic problems

- **Producer–Consumer:** producer adds items to a bounded buffer and consumer removes them; synchronization prevents overflow, underflow and races.
- **Readers–Writers:** coordinates multiple readers and writers accessing shared data.

- **Dining Philosophers:** illustrates deadlock and starvation risks when competing for shared resources.

Chapter 5: Deadlocks

A **deadlock** occurs when a set of processes are permanently waiting for resources held by one another.

Four necessary conditions (Coffman conditions)

- **Mutual Exclusion:** at least one resource is non-shareable.
- **Hold and Wait:** a process holds resources while requesting more.
- **No Preemption:** resources cannot be forcibly taken away under the relevant model.
- **Circular Wait:** a circular chain of processes waits for resources held by the next process.

Deadlock handling methods

- **Prevention:** design the system so at least one necessary condition cannot hold.
- **Avoidance:** allocate resources only when the resulting state is safe; the Banker's algorithm is a classic method.
- **Detection and recovery:** allow deadlocks, detect them, then recover by terminating processes or preempting resources where possible.
- **Ignore:** some general-purpose systems rarely use explicit deadlock handling for selected resource types.

Safe state

A state is safe if there is at least one order in which every process can obtain its remaining resources, finish, and release resources. An unsafe state is not necessarily already deadlocked.

Banker's algorithm overview

- Need = Maximum – Allocation.
- Work starts as Available; Finish is initially false for unfinished processes.
- Find a process whose Need is no greater than Work; pretend it finishes and add its Allocation to Work.
- Repeat. If all processes can finish, the state is safe.

Chapter 6: Memory Management

Memory management tracks main memory and allocates it to processes while supporting protection and efficient use.

Address binding and MMU

- **Logical/virtual address:** address generated by the CPU.
- **Physical address:** actual location in main memory.
- **Memory Management Unit (MMU):** hardware that translates addresses and enforces relevant access protections.

Contiguous memory allocation

- **Fixed partitioning:** memory is divided into fixed-size partitions; may cause internal fragmentation.
- **Variable partitioning:** partitions are created according to allocation needs; may cause external fragmentation.
- **First Fit:** use the first adequate free block.
- **Best Fit:** use the smallest adequate free block.
- **Worst Fit:** use the largest available free block.

Fragmentation

Internal fragmentation is wasted space inside an allocated block. **External fragmentation** is free memory split into small non-contiguous holes. Compaction may reduce external fragmentation where relocation is supported.

Paging

Paging divides virtual memory into fixed-size **pages** and physical memory into same-size **frames**. A page table maps pages to frames. Paging avoids external fragmentation in the physical allocation of pages, but may cause internal fragmentation, commonly in the final page.

Segmentation

Segmentation divides a program into logical, variable-sized units such as code, data and stack. A segment table commonly stores a base and limit for each segment. Segmentation can reflect program structure but can suffer external fragmentation.

Paging vs Segmentation

Paging	Segmentation
Fixed-size pages and frames.	Variable-size logical segments.
Usually transparent to the programmer.	Reflects logical program units.
No external fragmentation from page placement; internal fragmentation is possible.	External fragmentation is possible.

Chapter 7: Virtual Memory

Virtual memory lets a process use an address space that may be larger than the available physical RAM by keeping only some parts in RAM.

Demand paging and page fault

In **demand paging**, a page is loaded when it is first needed. A **page fault** occurs when the referenced page is not currently present in RAM; the OS checks validity, locates the page on storage, obtains a frame (possibly evicting another page), loads it and updates the page table.

Page replacement algorithms

- **FIFO:** evicts the page that entered memory earliest; can exhibit Belady's anomaly.
- **Optimal:** evicts the page whose next use is farthest in the future; theoretical benchmark, not generally implementable exactly.
- **LRU:** evicts the page that has not been used for the longest time in the past.
- **Second Chance / Clock:** FIFO-like method that uses a reference bit to give recently referenced pages another chance.

Thrashing

Thrashing occurs when excessive page faults cause the system to spend much of its time moving pages rather than executing useful work. Causes include too few frames for the active working sets. Possible responses include reducing the degree of multiprogramming or allocating more frames.

TLB

A **Translation Lookaside Buffer** is a fast cache of recent page-table translations. A TLB hit avoids a full page-table lookup; a miss requires additional translation work.

Chapter 8: File System Management

A file system organizes and stores files on storage devices and provides naming, access, metadata and protection.

File concepts and attributes

- File attributes may include name, type, size, location, owner, permissions and timestamps.

- Common operations: create, open, read, write, seek, close, rename and delete.
- Directories organize files and may be single-level, two-level, tree-structured or graph-based.

File access methods

- **Sequential access:** records are processed in order.
- **Direct / Random access:** access a particular location or block directly.
- **Indexed access:** an index helps locate records or blocks.

File allocation methods

- **Contiguous allocation:** consecutive disk blocks; fast access but difficult growth and possible external fragmentation.
- **Linked allocation:** each block points to the next; easy growth but inefficient random access and pointer overhead.
- **Indexed allocation:** an index block stores pointers to file blocks; supports direct access with index overhead.

Free-space management

Common methods include bitmaps/bit vectors, linked free lists, grouping and counting. File-system reliability can use journaling, consistency checking and backups.

Mounting

Mounting makes a file system available at a directory (mount point) in the existing directory hierarchy.

Chapter 9: I/O and Disk Management

The OS manages communication between software and devices, usually through device drivers and interrupt handling.

I/O techniques

- **Programmed I/O:** CPU repeatedly checks or controls device operations.
- **Interrupt-driven I/O:** device signals the CPU when attention is needed.
- **DMA (Direct Memory Access):** a controller transfers data between a device and memory with limited CPU involvement.

Buffering, caching and spooling

- **Buffering:** temporarily holds data during transfer between components.
- **Caching:** stores copies of frequently used data for faster access.
- **Spooling:** queues jobs for a device, such as print jobs, so multiple applications can submit work.

Disk scheduling algorithms

- **FCFS:** serves requests in arrival order.
- **SSTF:** selects the request closest to the current head position; may starve distant requests.
- **SCAN:** disk head moves in one direction servicing requests, then reverses.
- **C-SCAN:** services in one direction and returns to the beginning without servicing on the return.
- **LOOK / C-LOOK:** similar to SCAN / C-SCAN but reverses or wraps at the last pending request rather than the physical disk end.

Chapter 10: Protection and Security

Protection controls how processes and users access system resources. Security defends systems and data against threats.

Core concepts

- **Authentication:** verifies identity, for example with a password or multi-factor authentication.
- **Authorization:** determines which actions an authenticated identity may perform.
- **Least privilege:** grants only the permissions needed for a task.
- **Access control:** permissions, access-control lists and capability-based mechanisms regulate use of resources.
- **Encryption:** protects confidentiality by transforming data using cryptographic keys.
- **Auditing and logging:** records activities for monitoring and investigation.

Threats and safeguards

- Threats include malware, unauthorized access, privilege escalation, data theft and denial of service.
- Use updates, secure configuration, backups, strong authentication, least privilege and monitoring.
- A firewall filters network traffic according to rules; antivirus/endpoint protection detects or blocks malicious activity.

Chapter 11: Linux / UNIX Basics

Linux is a family of Unix-like operating systems. A shell interprets commands and launches programs.

Common commands

Command	Purpose
pwd	Show current directory
ls	List directory contents
cd	Change directory
mkdir	Create a directory
rmdir	Remove an empty directory
touch	Create an empty file or update timestamps
cp	Copy files or directories
mv	Move or rename files
rm	Remove files (use carefully)
cat	Display or concatenate file contents
grep	Search text using a pattern
chmod	Change permissions
ps	Display process information
kill	Send a signal to a process
man	Open a command's manual page

File permissions

Unix-like permissions commonly specify read (r), write (w) and execute (x) for the owner, group and others. For example, **chmod 755 file** commonly gives the owner rwx and group/others r-x on a regular file.

Chapter 12: Virtualization and Modern OS Concepts

Virtualization

Virtualization allows multiple isolated virtual machines to run on one physical host. A **hypervisor** creates and manages VMs. Type 1 hypervisors run directly on hardware; Type 2 hypervisors run on a host OS.

Containers

Containers isolate applications while sharing the host OS kernel. They are generally lighter than full VMs, but they do not provide the same separate-kernel model as virtual machines.

Cloud and mobile operating systems

Cloud systems use networked, virtualized and distributed resources. Mobile OSs add power management, app sandboxing, touch interfaces, permission controls and device-specific services.

Booting process

- Firmware (BIOS/UEFI) initializes hardware and selects a boot device.
- A bootloader loads the OS kernel.
- The kernel initializes memory, drivers and core subsystems.
- The system starts essential services and presents a login screen or user interface.

Chapter 13: Important Differences for Exams

Multiprogramming vs Multitasking

- **Multiprogramming:** several jobs stay in memory; CPU switches when a job waits, aiming to improve utilization.
- **Multitasking:** CPU time is shared among tasks, often with preemption, to support interactive use.

Deadlock vs Starvation

- **Deadlock:** a group of processes wait indefinitely for resources held by each other.
- **Starvation:** a process waits indefinitely because resources or CPU time keep being given to others; aging can help in priority scheduling.

Mutex vs Semaphore

- **Mutex:** typically has ownership; the thread that locks it is expected to unlock it.
- **Semaphore:** a counter-based signaling mechanism; it may coordinate access to one or more resource instances.

Paging vs Virtual Memory

- **Paging:** a memory-management technique that divides address spaces and physical memory into pages and frames.
- **Virtual memory:** a broader technique that gives processes virtual address spaces and may keep portions on secondary storage; paging is commonly used to implement it.

Program vs Process

- **Program:** passive executable instructions.
- **Process:** a running instance of a program with state and allocated resources.

Chapter 14: Quick Revision – Key Definitions

- **Operating System:** system software that manages resources and provides services to applications.
- **Process:** a program in execution.
- **Thread:** a basic unit of CPU execution within a process.
- **Scheduler:** OS component that selects work to run.
- **Semaphore:** synchronization primitive used to coordinate access or signal events.
- **Deadlock:** indefinite waiting caused by a circular or otherwise unsatisfiable resource dependency.
- **Paging:** fixed-size division of virtual memory into pages and physical memory into frames.
- **Page fault:** event raised when a required page is not currently resident in RAM.
- **Thrashing:** excessive paging that severely reduces useful execution.

- **File system:** method and data structures used to organize and access files.
- **DMA:** data transfer between a device and memory with reduced CPU involvement.
- **System call:** interface for requesting OS kernel services.
- **Context switch:** saving one execution context and restoring another.
- **Race condition:** outcome depends on timing/order of concurrent operations.

Chapter 15: Practice Questions

Short-answer questions

- Define an operating system and explain any five functions.
- Explain the different types of operating systems with examples.
- Draw and explain the process-state diagram.
- What is a PCB? List its contents.
- Compare process and thread.
- Explain FCFS, SJF, Priority and Round Robin scheduling.
- Define race condition and critical section. State the three requirements of a correct solution.
- Explain semaphores and monitors.
- State the four necessary conditions for deadlock.
- Explain the Banker's algorithm and safe state.
- Differentiate internal and external fragmentation.
- Explain paging, segmentation and virtual memory.
- Compare FIFO, Optimal and LRU page replacement.
- What is thrashing? How can it be reduced?
- Explain file allocation methods.
- Compare programmed I/O, interrupt-driven I/O and DMA.
- Explain disk scheduling algorithms.
- Differentiate authentication and authorization.
- Write any ten Linux commands with their uses.
- Explain virtualization and compare VMs with containers.

Numerical practice checklist

- CPU scheduling: Gantt chart, CT, TAT, WT and averages.
- Page replacement: count page faults for a given reference string and frame count.
- Disk scheduling: calculate total head movement from the given request queue and initial head position.
- Banker's algorithm: calculate Need = Maximum – Allocation and find a safe sequence.

Final exam tip

Learn each definition, draw diagrams where required, prepare comparison tables, and practise numerical questions. Your university's official syllabus and past papers should determine which chapters and algorithms need the most attention.